

CEA Reserve System Map

1. Architecture: today versus target

The left side is what runs now. Every route reads from two static TypeScript modules and holds edits in React state, so a refresh discards each submission and approval. The right side is the layered system the backlog describes, with the issue number that delivers each part.

 exists in the repo  planned, with the GitHub issue that adds it

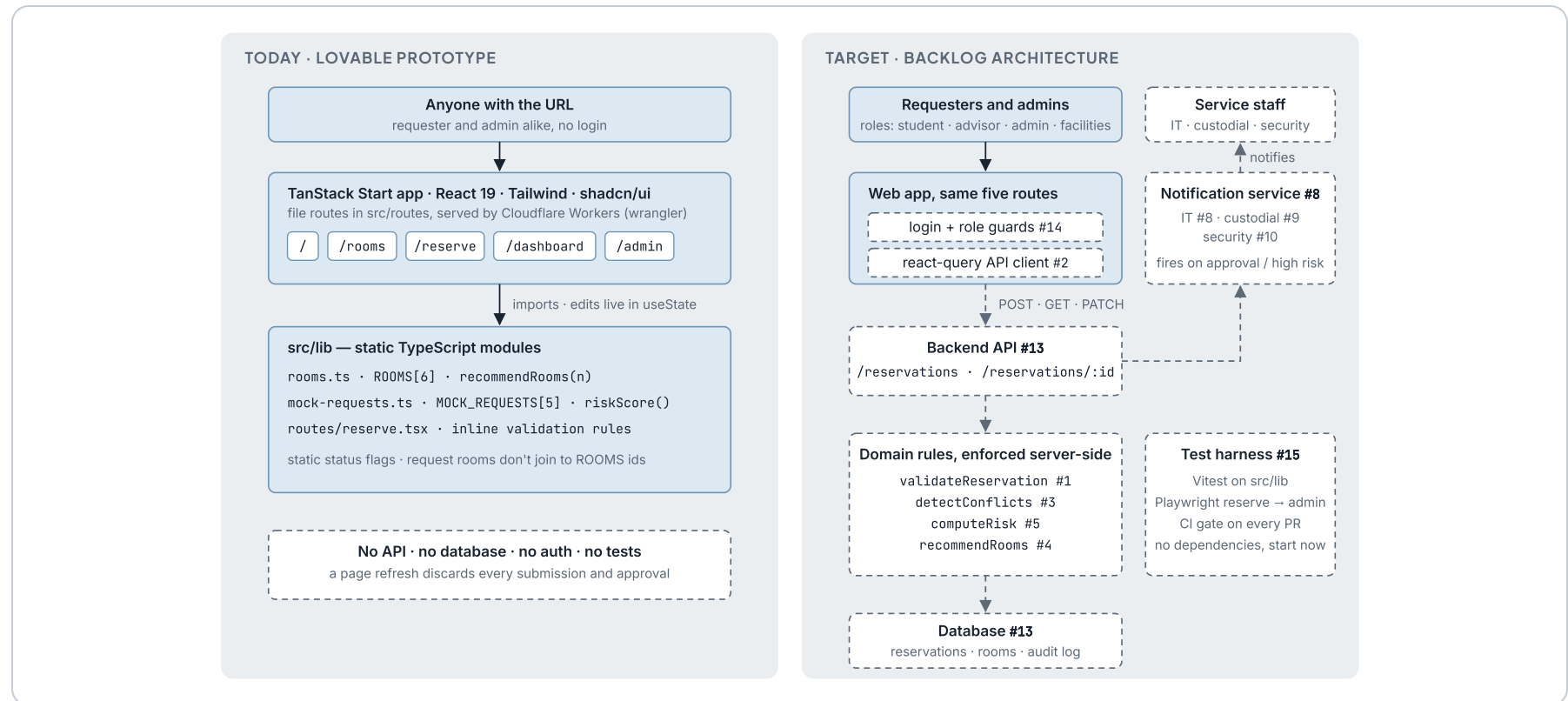


Figure 1. The prototype stops at the browser. The target moves every rule the client applies today behind an API so it cannot be bypassed, and adds the three service integrations the README already promises.

2. Life of a reservation request

This is the path a request takes through the app, with the rules the code applies at each step. The client-side checks and room suggestions work today. Persisting the submission and notifying staff are planned.

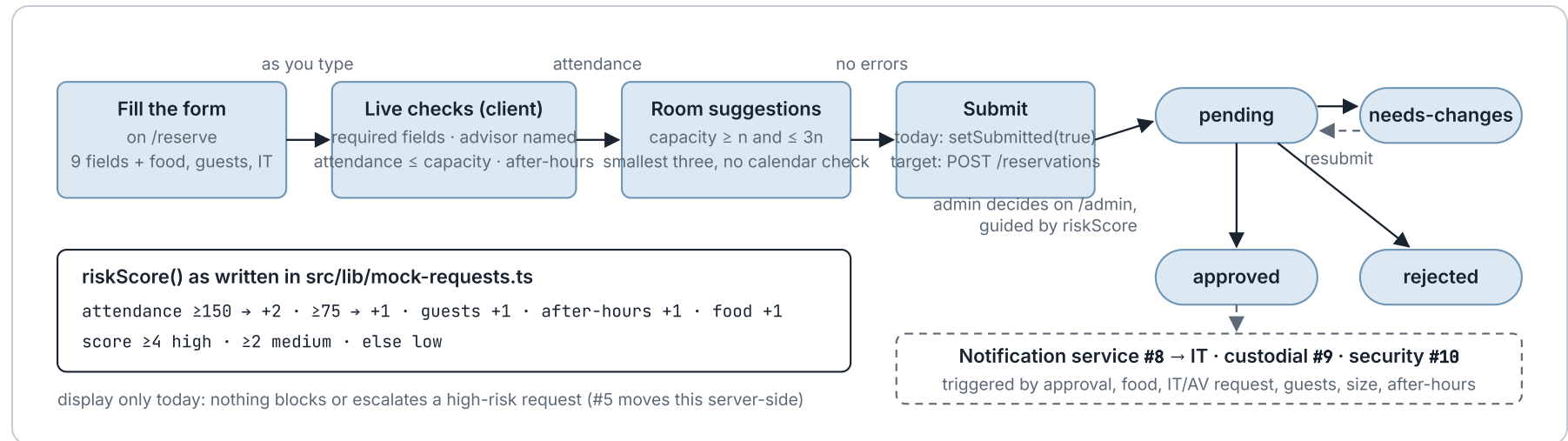


Figure 2. Solid boxes and arrows run in the browser today. The dashed resubmit edge, the persistence step, and the notification fan-out exist only as tickets.

3. Ticket dependency graph

Every issue carries a priority and a "depends on" note. Read left to right: a ticket can start once everything pointing into it is merged. The persistent API is the root of almost everything, which is why the board says to start there.

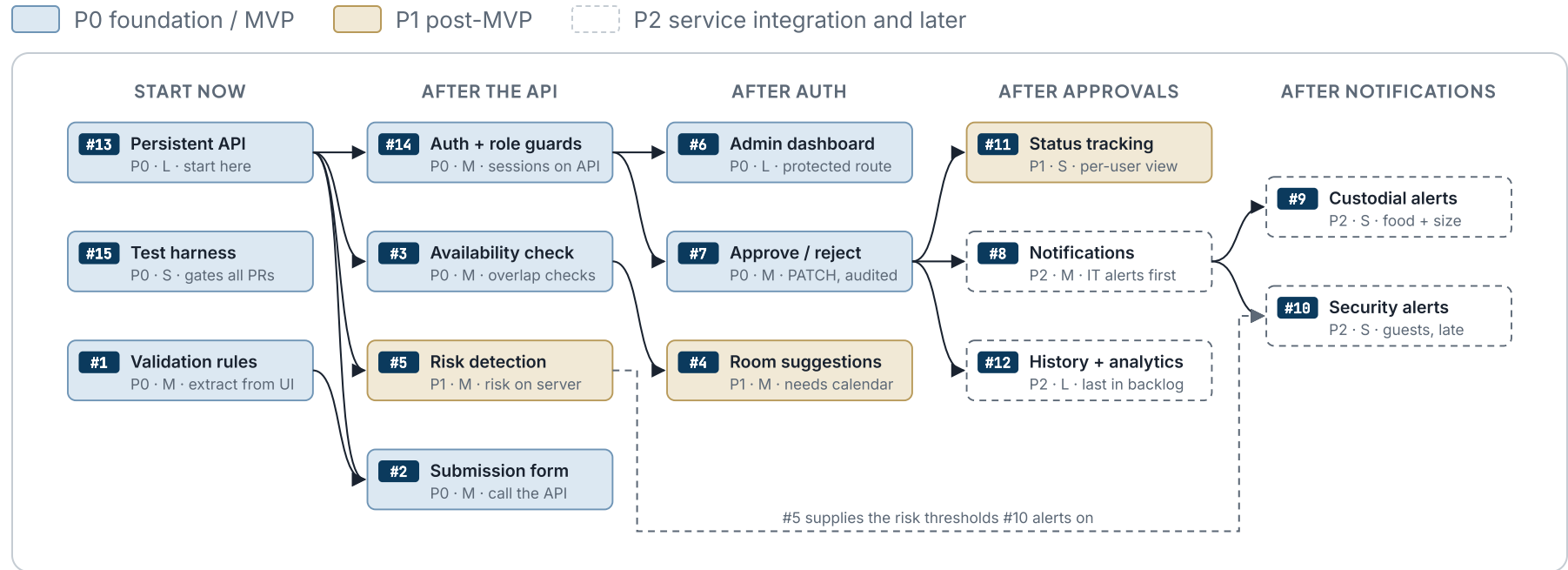


Figure 3. Edges are the direct dependencies from each issue's backlog notes. Edges that only repeat "depends on #13" through another ticket are omitted; every ticket right of the first column already needs the API.

4. Routes and what feeds them

ROUTE	WHAT IT SHOWS	READS FROM TODAY	WRITES
/	Landing page with feature pitch and links	static copy	none
/rooms	Six room cards with capacity, features, availability flag	ROOMS	none
/reserve	Reservation form, live validation, up to three suggested rooms, confirmation panel	ROOMS , recommendRooms	local state only
/dashboard	Status counts and list of all mock requests	MOCK_REQUESTS	none
/admin	Filterable request table with risk level, approve and reject buttons	MOCK_REQUESTS , riskScore	local state only

5. What the audit flags

Open admin surface

- Anyone can approve or reject on /admin .
- Fixed by #14, which needs #13 for server-side sessions.

Rules live in the UI

- Capacity and after-hours checks sit inline in the reserve page.
- Risk score is display only and never enforced.

Docs ahead of code

- README promises an API, conflict detection, notifications, and AWS.
- Package is still named tanstack_start_ts , with both Bun and npm lockfiles.

Zero tests

- No runner, no test script, no CI workflow.
- #15 is small, has no dependencies, and unblocks the definition of done for every other ticket.

Sources: the repository at joniabeje/backend-builders on main (commit e47a61d), README.md, DEBT_AND_RISK.md, and GitHub issues #1 through #15 with their backlog notes finalized 2026-09-23. Ticket labels F1 to F16 are the original feature codes.